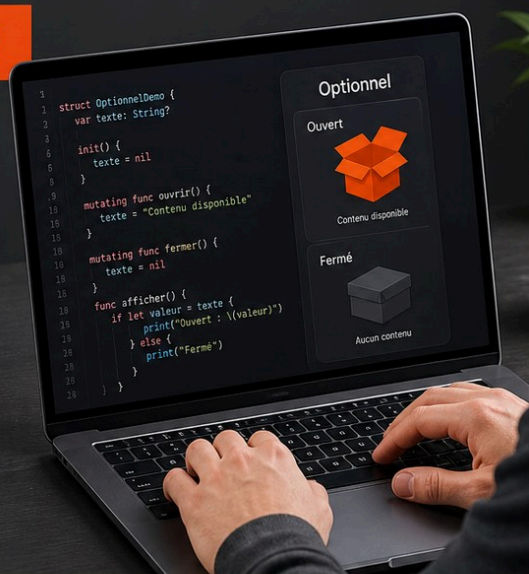




POUR DÉBUTANTS

APPRENEZ À CRÉER VOS PREMIÈRES
APPLICATIONS iOS AVEC SWIFT

ÉDITION
2024



FORMATION SWIFT - LES BASES

Swift

Apprendre le langage Apple pour iOS et
macOS, pas à pas.

DanielCraft - Livre debutant - Pack Apple

Sommaire

Clique un chapitre ou un sous-chapitre pour y aller.

1. [C'est quoi Swift ?](#)

- [Le langage en une phrase](#)
- [Pourquoi apprendre Swift ?](#)
- [A quoi ca ressemble ?](#)
- [Petite histoire](#)
- [En vrai](#)
- [A retenir](#)

2. [Installer Swift](#)

- [Sur Mac : Xcode](#)
- [Creer un projet](#)
- [Sur Linux / Windows](#)
- [Petite histoire](#)
- [A retenir](#)

3. [Premier programme](#)

- [Hello World](#)
- [Affichage formate](#)
- [Les commentaires](#)
- [Petite histoire](#)
- [A retenir](#)

4. [Les variables](#)

- [let vs var](#)
- [Types explicites](#)
- [Constantes](#)
- [Conventions](#)
- [Petite histoire](#)
- [A retenir](#)

5. [Les types de donnees](#)

- [Types numeriques](#)
- [Texte](#)
- [Conversion](#)
- [Tuples](#)
- [Petite histoire](#)
- [A retenir](#)

6. [Les conditions](#)

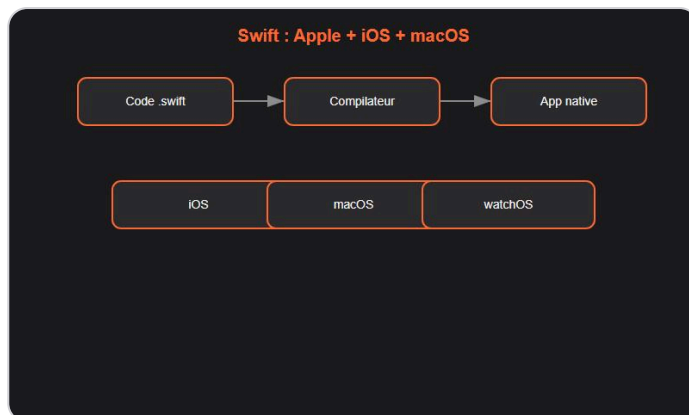
- [if / else](#)
- [if comme expression \(Swift 5.9+\)](#)
- [switch](#)
- [Petite histoire](#)
- [A retenir](#)

7. [Les boucles](#)

- [for-in](#)
 - [Parcourir une collection](#)
 - [while et repeat-while](#)
 - [break et continue](#)
 - [A retenir](#)
8. [Les fonctions](#)
- [Declarer une fonction](#)
 - [Parametres avec labels](#)
 - [Parametres par default](#)
 - [Closures](#)
 - [A retenir](#)
9. [Les optionals](#)
- [Le probleme du nil](#)
 - [Deballage securise](#)
 - [Guard let](#)
 - [Nil coalescing](#)
 - [Force unwrap \(a eviter\)](#)
 - [Petite histoire](#)
 - [A retenir](#)
10. [Structs et classes](#)
- [Struct \(type valeur\)](#)
 - [Class \(type reference\)](#)
 - [Quand choisir ?](#)
 - [A retenir](#)
11. [Les enums](#)
- [Enum simple](#)
 - [Enum avec valeurs associees](#)
 - [switch sur enum](#)
 - [Raw values](#)
 - [A retenir](#)
12. [Collections](#)
- [Array](#)
 - [Dictionary](#)
 - [Set](#)
 - [Operations fonctionnelles](#)
 - [A retenir](#)
13. [Gerer les erreurs](#)
- [throw et Error](#)
 - [try / catch](#)
 - [try? et try!](#)
 - [A retenir](#)
14. [Mini-projet : liste de courses CLI](#)

- [Objectif](#)
 - [A retenir](#)
15. [Ce qu'il faut retenir](#)
- [Suite](#)
16. [Atelier : variables](#)
- [Exercice 1 : carte d'identite](#)
 - [Exercice 2 : optional](#)
 - [Defi : temperature](#)
17. [Atelier : fonctions](#)
- [Exercice 1 : salutation](#)
 - [Exercice 2 : moyenne](#)
 - [Exercice 3 : division securisee](#)
18. [Erreurs classiques](#)
- [1. Force unwrap avec !](#)
 - [2. Confondre struct et class](#)
 - [3. Oublier break dans switch](#)
 - [4. Mutable vs immutable collection](#)
 - [5. Optional non deballe](#)
19. [Bonnes pratiques](#)
- [Idiomes Swift](#)
 - [Formatage](#)
 - [Tests](#)
 - [A retenir](#)
20. [Quiz Swift - Les bases](#)
21. [Bravo !](#)
- [La suite](#)

C'est quoi Swift ?



Swift : langage officiel Apple.



Couverture Swift.

Le langage en une phrase

Swift est le langage moderne d'Apple pour iOS, iPadOS, macOS, watchOS et tvOS. Il est rapide, sûr et conçu pour être agréable à écrire.

Pourquoi apprendre Swift ?

Swift remplace Objective-C sur l'écosystème Apple. Il est utilisé par des millions d'applications sur l'App Store. Apple le pousse aussi côté serveur et Linux.

> **Astuce DanielCraft** - Si tu vises iOS ou le développement Apple, Swift est incontournable.

Nora découvre Swift en voulant créer sa première app iPhone. Elle écrit 10 lignes et voit un résultat concret dans le simulateur.

A quoi ça ressemble ?

```
let nom = "Lea"
print("Salut \(nom), bienvenue en Swift !")
```

Petite histoire

Apple présente Swift en 2014 lors de la WWDC. L'objectif : un langage plus sûr, plus rapide et plus moderne qu'Objective-C, tout en restant interopérable.

En vrai

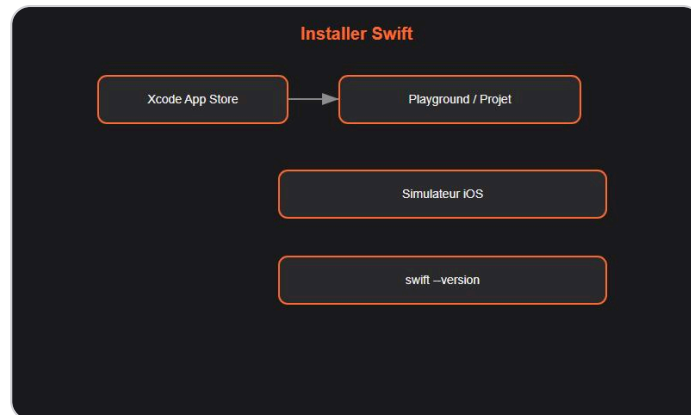
Sam developpe une app de suivi de budget. Max cree un widget macOS. Nora automatise des taches avec un script Swift en ligne de commande.

> **Piege** - Swift ressemble parfois a Kotlin ou Python, mais ses optionals et ses structs ont leurs propres regles.

A retenir

- Swift = langage officiel Apple.
- `let` immutable, `var` mutable.
- Ideal pour apps mobiles et desktop Apple.

Installer Swift



Xcode + Playground.

Sur Mac : Xcode

1. Installe **Xcode** depuis l'App Store.
2. Ouvre Xcode et accepte la licence.
3. Verifie dans le terminal :

```
swift --version
```

Creer un projet

- **App iOS** : File > New > Project > App (SwiftUI ou UIKit).
- **Script CLI** : File > New > Project > macOS > Command Line Tool.

Sur Linux / Windows

Swift est aussi disponible sur Linux. Sur Windows, l'experience est plus limitee ; pour iOS, un Mac reste necessaire.

> **Astuce DanielCraft** - Pour debuter, un playground Xcode suffit pour tester du code sans creer un projet complet.

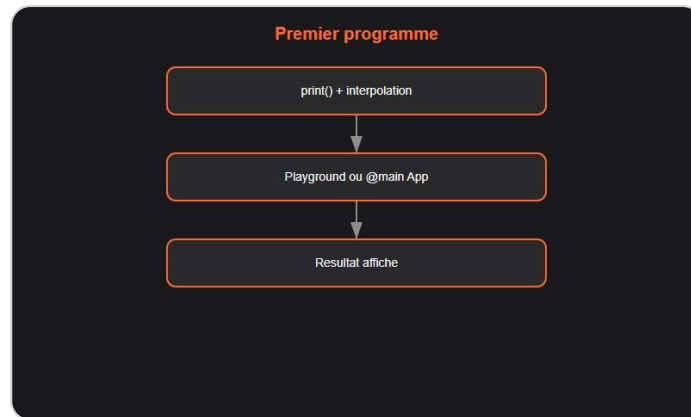
Petite histoire

Max installe Xcode, cree un Playground, tape `print("Hello")` et voit le resultat instantanement.

A retenir

- Xcode = IDE + simulateur + outils Apple.
- `swift --version` pour verifier l'installation.
- Playground ideal pour experimenter.

Premier programme



`print()` et interpolation.

Hello World

```
print("Bonjour le monde !")
```

Dans un projet App, le point d'entree est `@main` sur une struct `App`.

Affichage formate

```
let nom = "Sam"
let age = 28
print("Je suis \(nom), \(age) ans.")
```

Les commentaires

```
// Commentaire sur une ligne
/* Commentaire
   sur plusieurs lignes */
```

Petite histoire

Nora ouvre un Playground, écrit trois lignes, exécute. Le résultat s'affiche dans le panneau de droite.

A retenir

- `print()` pour afficher.
- Interpolation avec `\(variable)`.
- Playground pour tester rapidement.

Les variables



let / var.

let vs var

```
let nom = "Max"      // Immutable
var score = 0        // Mutable
score += 10
print(score)         // 10
```

> **Astuce DanielCraft** - Prefere `let` par default. Utilise `var` seulement si la valeur change.

Types explicites

```
let age: Int = 25
let prix: Double = 19.99
let actif: Bool = true
```

Constantes

```
let tva = 0.20
let maxJoueurs = 100
```

Conventions

- camelCase pour variables et fonctions.
- PascalCase pour types, structs, classes, enums.

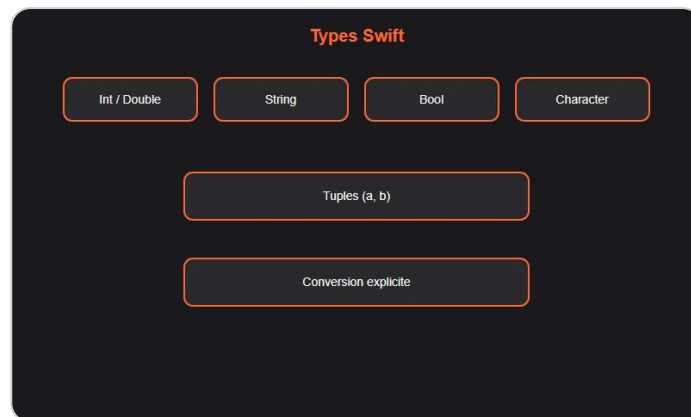
Petite histoire

Sam declare `let budget = 1800` puis essaie de le modifier. Xcode refuse immédiatement.

A retenir

- `let` = immutable, `var` = mutable.
- Inference de type par default.
- camelCase / PascalCase.

Les types de donnees



Int, Double, String, Bool.

Types numeriques

Type	Exemple	Usage
Int	42	Entier (default)
Double	3.14	Flottant (default)
Float	3.14	Flottant simple precision
Bool	true	Booleen

Texte

```
let texte: String = "Bonjour"
let lettre: Character = "A"
```

Conversion

```
let i = Int("42")           // Optional Int?
let s = String(42)
```

Tuples

```
let point = (10, 20)
print(point.0) // 10
print(point.1) // 20
```

Petite histoire

Nora additionne un `Int` et un `Double`. Swift refuse. Elle convertit avec `Double(entier)`.

A retenir

- `Int`, `Double`, `String`, `Bool` sont les types courants.

- Conversions explicites.
- Tuples pour regrouper plusieurs valeurs.

Les conditions



if + switch exhaustif.

if / else

```
let age = 17
if age >= 18 {
    print("Majeur")
} else {
    print("Mineur")
}
```

if comme expression (Swift 5.9+)

```
let statut = age >= 18 ? "Majeur" : "Mineur"
```

switch

```
let jour = "lundi"
switch jour {
case "lundi", "mardi":
    print("Debut de semaine")
case "vendredi":
    print("Presque le weekend")
default:
    print("Autre jour")
}
```

`switch` doit etre exhaustif en Swift.

> **Astuce DanielCraft** - `switch` en Swift est tres puissant : ranges, tuples, patterns.

Petite histoire

Max ecrit un `switch` sur une enum `Direction`. Le compilateur lui signale le cas manquant.

A retenir

- `if / else` classique.
- `switch` exhaustif et puissant.

- Pas de fallback implicite.

Les boucles



Coder en Swift au quotidien.



for-in et while.

for-in

```
for i in 0...4 {  
    print(i) // 0, 1, 2, 3, 4  
}  
  
for i in 0...<5 {  
    print(i) // 0, 1, 2, 3, 4  
}
```

Parcourir une collection

```
let fruits = ["pomme", "banane", "cerise"]  
for fruit in fruits {  
    print(fruit)  
}
```

while et repeat-while

```
var n = 0
while n < 5 {
    print(n)
    n += 1
}

repeat {
    print(n)
    n -= 1
} while n > 0
```

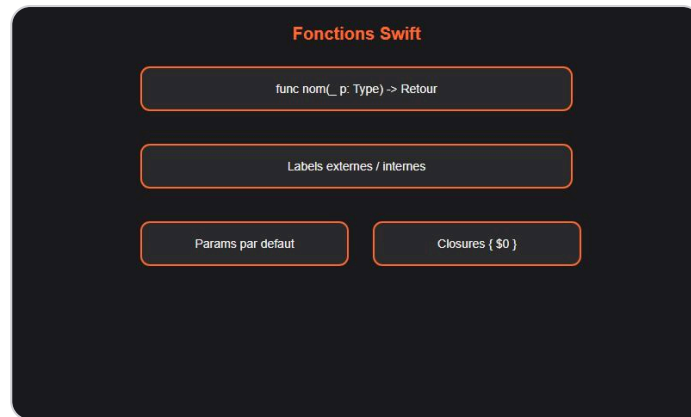
break et continue

```
for i in 0...10 {
    if i == 5 { break }
    if i % 2 == 0 { continue }
    print(i)
}
```

A retenir

- `0...4` inclusif, `0.. $<5$` exclusif.
- `for-in` pour parcourir collections.
- `repeat-while` teste apres execution.

Les fonctions



Fonctions et closures.

Declarer une fonction

```
func saluer(prenom: String) {
    print("Bonjour \(prenom) !")
}

func additionner(a: Int, b: Int) -> Int {
    return a + b
}
```

Parametres avec labels

```
func retirer(de compte: Double, montant: Double) -> Double {
    return compte - montant
}

retirer(de: 1000, montant: 50)
```

Parametres par default

```
func saluer(prenom: String, message: String = "Bonjour") {
    print("\(message) \(prenom) !")
}
```

Closures

```
let nombres = [3, 1, 4, 1, 5]
let doubles = nombres.map { $0 * 2 }
let pairs = nombres.filter { $0 % 2 == 0 }
```

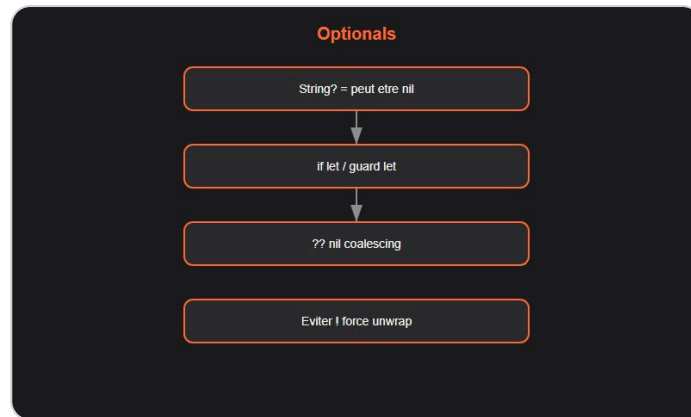
> **Astuce DanielCraft** - `$0`, `$1` sont les raccourcis pour les parametres de closure.

A retenir

- `func nom(_ param: Type) -> Retour`.
- Labels externes et internes.

- Closures avec `{ $0 }`.

Les optionals



Optionals : if let, guard, ??.

Le probleme du nil

Swift utilise `Optional` pour représenter l'absence de valeur :

```
var nom: String? = nil
nom = "Lea"
```

Deballage securise

```
if let valeur = nom {
    print(valeur.count)
}
```

Guard let

```
func afficherLongueur(_ texte: String?) {
    guard let texte = texte else { return }
    print(texte.count)
}
```

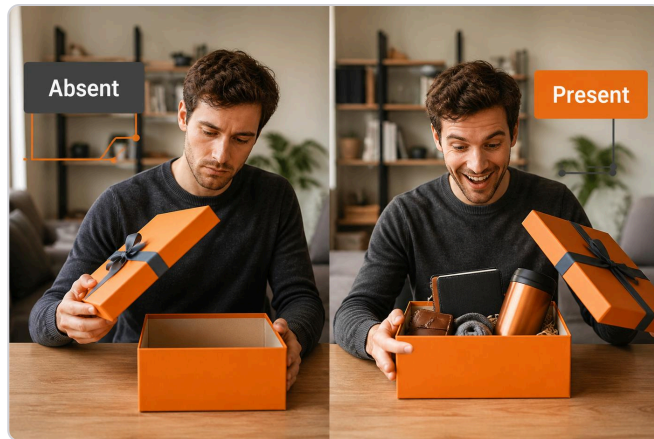
Nil coalescing

```
let affichage = nom ?? "Inconnu"
```

Force unwrap (a eviter)

```
let longueur = nom!.count // Crash si nil !
```

> **Piege** - `!` force le deballage. Reserve-le aux cas ou tu es certain.



Visualiser les optionals.

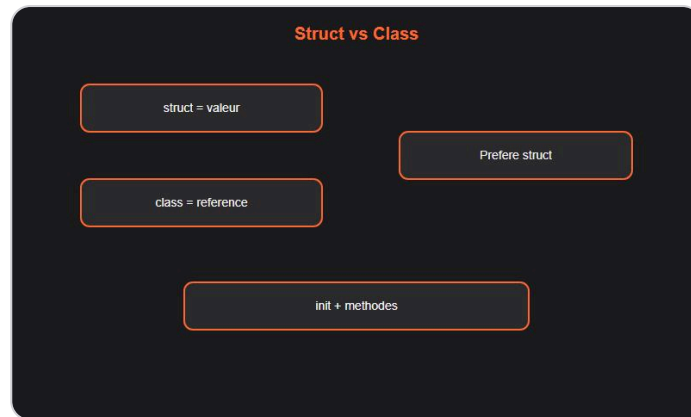
Petite histoire

Nora recoit un champ optional d'une API. Elle utilise `if let` et evite tout crash.

A retenir

- `Type?` = optional (peut etre nil).
- `if let`, `guard let`, `??` pour gerer nil.
- Evite `!` en production.

Structs et classes



Struct vs class.

Struct (type valeur)

```
struct Animal {
    var nom: String
    var age: Int

    func sePresenter() -> String {
        "Je suis \(nom), \(age) ans."
    }
}

let chat = Animal(nom: "Felix", age: 3)
print(chat.sePresenter())
```

Class (type reference)

```
class Compte {
    var solde: Double
    init(solde: Double) { self.solde = solde }
    func depot(_ montant: Double) { solde += montant }
}
```

Quand choisir ?

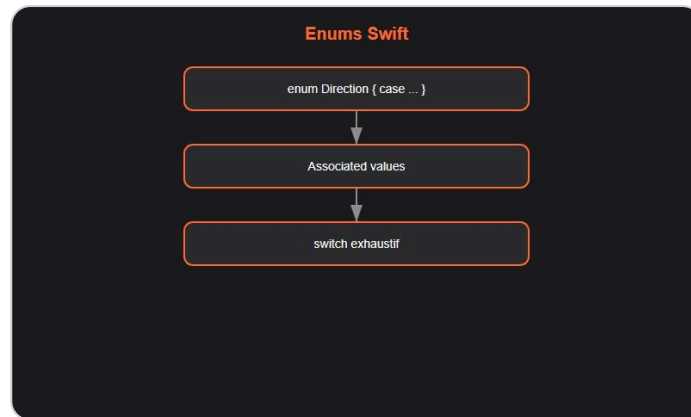
- **Struct** : donnees simples, copie par valeur (prefere en Swift).
- **Class** : heritage, identite partagee, reference.

> **Astuce DanielCraft** - Swift prefere les structs. Utilise une class seulement si tu as besoin d'heritage ou de reference.

A retenir

- Struct = valeur, Class = reference.
- Swift prefere les structs.
- `init` pour le constructeur.

Les enums



Enums et associated values.

Enum simple

```
enum Direction {
    case nord, sud, est, ouest
}

let dir = Direction.nord
```

Enum avec valeurs associees

```
enum Message {
    case quitter
    case texte(String)
    case position(x: Int, y: Int)
}

let msg = Message.texte("Bonjour")
```

switch sur enum

```
switch dir {
case .nord: print("Vers le nord")
case .sud: print("Vers le sud")
case .est: print("Vers l'est")
case .ouest: print("Vers l'ouest")
}
```

Raw values

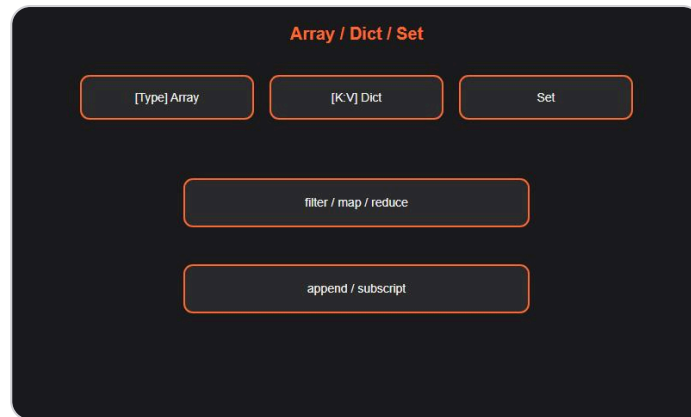
```
enum Jour: String {
    case lundi = "Lundi"
    case mardi = "Mardi"
}
```

A retenir

- `enum` pour les variantes.

- Valeurs associees pour porter des donnees.
- `switch` exhaustif sur les enums.

Collections



Array, Dictionary, Set.

Array

```
var fruits = ["pomme", "banane", "cerise"]
fruits.append("kiwi")
print(fruits.count) // 4
```

Dictionary

```
var ages = ["Lea": 28, "Sam": 22]
ages["Nora"] = 30
print(ages["Lea"] ?? 0)
```

Set

```
var unique = Set([1, 2, 3, 2, 1])
print(unique) // {1, 2, 3}
```

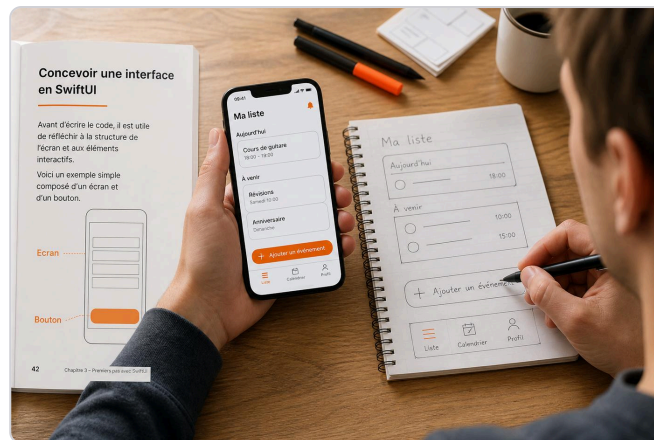
Operations fonctionnelles

```
let nombres = [3, 1, 4, 1, 5]
let sorted = nombres.sorted()
let filtres = nombres.filter { $0 > 2 }
let somme = nombres.reduce(0, +)
```

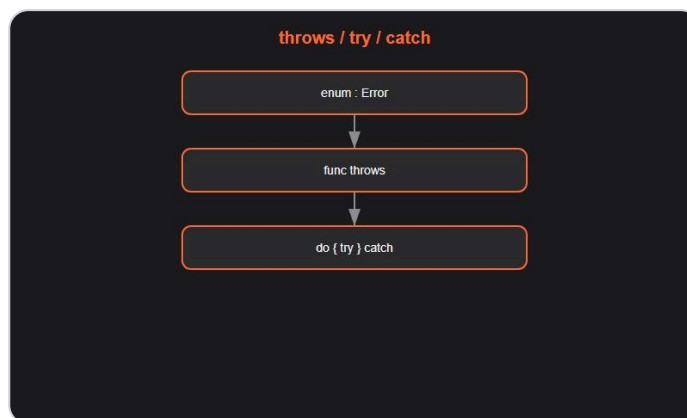
A retenir

- `[Type]` array, `[Cle: Valeur]` dictionary, `Set<Type>` ensemble.
- `append`, `filter`, `map`, `reduce`.

Gerer les erreurs



Swift pour iOS.



throws / try / catch.

throw et Error

```
enum MonErreur: Error {
    case soldeInsuffisant
    case montantInvalide
}

func retirer(solde: inout Double, montant: Double) throws {
    guard montant > 0 else { throw MonErreur.montantInvalide }
    guard montant <= solde else { throw MonErreur.soldeInsuffisant }
    solde -= montant
}
```


try / catch

```
var solde = 100.0
do {
    try retirer(solde: &solde, montant: 150)
} catch MonErreur.soldeInsuffisant {
    print("Solde insuffisant")
} catch {
    print("Erreur : \(error)")
}
```

try? et try!

```
let n = try? Int("abc")    // nil si echec
let m = try! Int("42")     // crash si echec
```

> **Astuce DanielCraft** - `throws` pour les erreurs attendues. `try?` pour simplifier quand nil suffit.

A retenir

- `throws` / `try` / `catch`.
- `enum` conforme a `Error`.
- `try?` retourne optional.

Mini-projet : liste de courses CLI



Mini-projet : liste de courses.

Objectif

Programme en ligne de commande pour ajouter, lister et supprimer des articles.

```

import Foundation

var courses: [String] = []

while true {
    print("\n1. Ajouter  2. Lister  3. Supprimer  4. Quitter")
    guard let choix = readLine()?.trimmingCharacters(in: .whitespaces) else { continue }

    switch choix {
    case "1":
        print("Article : ", terminator: "")
        if let article = readLine()?.trimmingCharacters(in: .whitespaces), !article.isEmpty {
            courses.append(article)
            print("Ajoute.")
        }
    case "2":
        if courses.isEmpty { print("Liste vide.") }
        else { for (i, a) in courses.enumerated() { print("  \(i + 1).  \(a)") } }
    case "3":
        print("Numero : ", terminator: "")
        if let idx = Int(readLine() ?? ""), idx >= 1, idx <= courses.count {
            courses.remove(at: idx - 1)
            print("Supprime.")
        }
    case "4":
        print("A bientôt !")
        exit(0)
    default:
        break
    }
}

```

A retenir

- Array + switch + readLine = CLI complet.
- `guard let` pour valider les entrees.

Ce qu'il faut retenir



Carte des notions Swift.

Concept	Resume
Variables	let / var
Types	Int , Double , String , Bool
Conditions	if , switch exhaustif
Boucles	for-in , while
Fonctions	labels, closures
Optionals	?, if let , ??
Structs/Classes	valeur vs reference
Enums	variantes + associated values
Collections	Array, Dictionary, Set
Erreurs	throws , try , catch

> **Astuce DanielCraft** - Swift est conçu pour la sécurité. Le compilateur te guide.

Suite

- SwiftUI pour les interfaces.
- Combine pour la programmation réactive.
- Async/await pour l'asynchrone.

Atelier : variables



Atelier : variables.

Exercice 1 : carte d'identite

```
let prenom = "Sam"
let age = 22
print("Je suis \(prenom), \(age) ans.")
```

Exercice 2 : optional

```
var pseudo: String? = nil
pseudo = "DevNora"
let affichage = pseudo ?? "Anonyme"
print(affichage)
```

Defi : temperature

```
let celsius = 37.0
let fahrenheit = celsius * 9 / 5 + 32
print("\(celsius)°C = \(fahrenheit)°F")
```

Atelier : fonctions



Atelier : fonctions.

Exercice 1 : salutation

```
func saluer(nom: String, heure: Int) -> String {  
    switch heure {  
        case ..<12: return "Bonjour \(nom) !"  
        case ..<18: return "Bon apres-midi \(nom) !"  
        default: return "Bonsoir \(nom) !"  
    }  
}
```

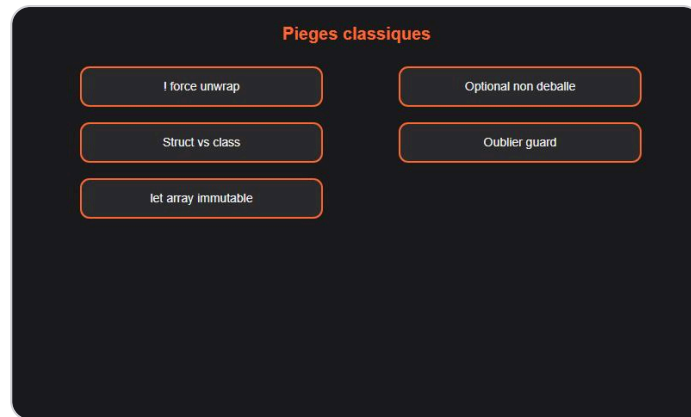
Exercice 2 : moyenne

```
func moyenne(_ notes: [Int]) -> Double {  
    guard !notes.isEmpty else { return 0 }  
    return Double(notes.reduce(0, +)) / Double(notes.count)  
}
```

Exercice 3 : division securisee

```
func diviser(_ a: Int, par b: Int) -> Int? {  
    guard b != 0 else { return nil }  
    return a / b  
}
```

Erreurs classiques



Pieges classiques Swift.

1. Force unwrap avec !

```
let nom: String? = nil
print(nom!.count) // Crash !
```

2. Confondre struct et class

Les structs sont copiees, les classes sont referencees.

3. Oublier break dans switch

Swift n'a pas de fallthrough implicite, mais attention aux cas vides.

4. Mutable vs immutable collection

```
let liste = [1, 2, 3]
liste.append(4) // Erreur : let = immutable
```

5. Optional non deballe

```
let n: Int? = 42
print(n + 1) // Erreur : deballe avec if let ou ??
```

> **Astuce DanielCraft** - Lis les messages du compilateur Xcode. Ils sont tres clairs.

Bonnes pratiques



Bonnes pratiques idiomatiques.

Idiomes Swift

- Prefere `let` a `var`.
- Prefere les `structs` aux classes.
- Utilise `guard` pour sortir tot d'une fonction.
- Nomme clairement : `isValid`, `hasItems`.

Formatage

- SwiftLint ou le formateur Xcode.
- 4 espaces d'indentation (standard Xcode).

Tests

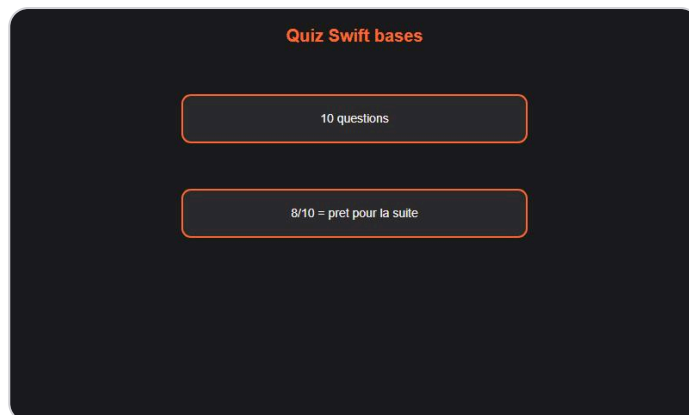
```
func testAddition() {  
    XCTAssertEqual(additionner(2, 3), 5)  
}
```

A retenir

- Code idiomatique Swift, pas du Java traduit.
- Tests unitaires avec XCTest.

QUIZ

Quiz Swift - Les bases



Quiz Swift bases.

- Q1.** Immutable en Swift : A) var B) let C) const D) final → **B**
- Q2.** Optional s'ecrit : A) Type! B) Type? C) Type* D) Type~ → **B**
- Q3.** Struct vs Class : A) Struct = reference B) Class = valeur C) Struct = valeur D) Identiques → **C**
- Q4.** ?? s'appelle : A) Safe call B) Nil coalescing C) Force unwrap D) Guard → **B**
- Q5.** switch doit etre : A) Optionnel B) Exhaustif C) Sans default D) Avec break → **B**
- Q6.** \$0 dans une closure : A) Erreur B) Premier parametre C) Retour D) Type → **B**
- Q7.** Langage officiel Apple : A) Objective-C B) Swift C) Rust D) Kotlin → **B**
- Q8.** throws indique : A) Fonction async B) Fonction qui peut lancer une erreur C) Constante D) Optional → **B**
- Q9.** Range inclusif : A) 0..<5 B) 0...4 C) 0-4 D) 0:5 → **B**
- Q10.** IDE principal : A) VS Code B) Xcode C) IntelliJ D) Eclipse → **B**
- > **Astuce DanielCraft** - 8/10 ou plus ? Pret pour Swift intermediaire (SwiftUI, async/await).

FINAL

Bravo !



Bravo. Tu codes en Swift.

Tu as terminé **Swift - Les bases**. Tu maîtrises variables, types, conditions, boucles, fonctions, optionals, structs, enums, collections et gestion d'erreurs.

La suite

- **Swift - Intermediaire** : SwiftUI, async/await, Combine, architecture MVVM.

> **Astuce DanielCraft** - Crée une mini-app dans le simulateur iOS. C'est la meilleure motivation.

Tu as les bases. Maintenant, ship.

Tu as les briques. Maintenant, ship.